5. Process Synchronization

Dr. Arun Kumar
Dept. of CSE, NIT Rourkela

Synchronization

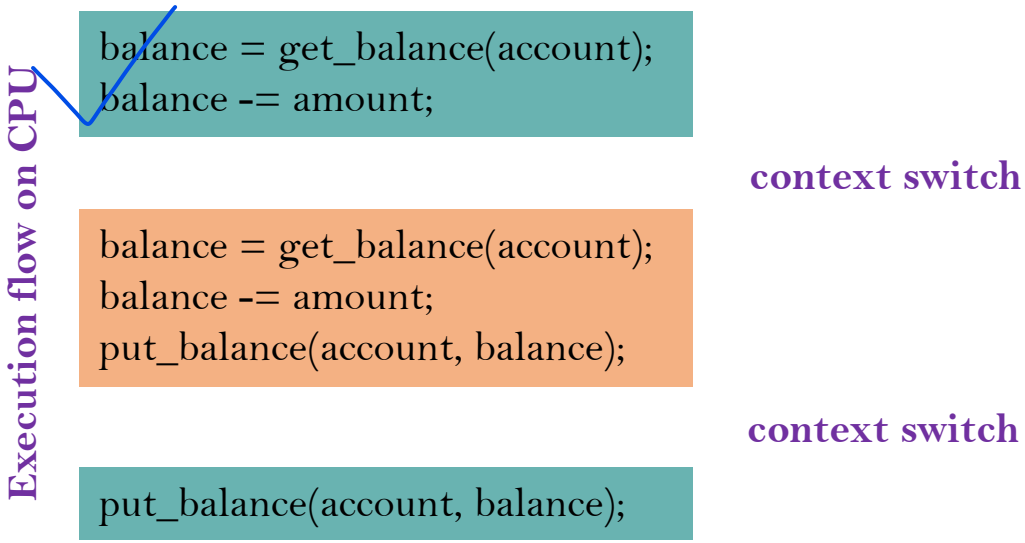
Example:

- Human Body – ***Walking***
- Printer
  - It's a ***shared resource*** but must be used in a ***non-sharable fashion.***
  - In other words – ***mutual exclusion***

Race Condition

- A condition when several processes *access* and *manipulate* the *same data-item*; and *final result* depends on the *order of access*.
- If three lines of code is running ***consecutively then no issue, issue is in case of context switch***.

```
int withdraw (account, amount) {
balance = get balance(account);
balance = balance - amount;
put balance(account, balance);
return balance;
}
```



Critical Section

- *Critical Section* is a *section of code* where some shared variable(s) is/are modified.

```
do {
        ...
        ...
    entry section

        critical section

    exit section

        reminder section
} while (TRUE);
```

Fig: General structure of a typical process with *critical section*

Solution to Critical-Section Problem

To manage critical sections, OS uses **synchronization techniques** (locks, semaphores, monitors). The solution ***must satisfy following criteria***:

- Mutual Exclusion (Mandatory) – *No two processes* may be simultaneously inside their *critical sections*.
- Progress (Mandatory) – If *no process* is executing in its critical section and there exist some processes that *wish to enter* their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
  - Example: P1, P2, P3 in Round Robin. May be ***P3 is not interested***.

Solution to Critical-Section Problem

Criteria

- Bounded Waiting (Optional) – *A bound must exist* on the *number of times* that other processes are allowed to enter their critical sections after a process has made *a request to enter* its critical section.
  - Example: To solve the ***starvation*** problem.

Two Processes: Solution – 1

- **`int turn`**
  - Shared between processes P_1 and P_2
  - initialized to 0 or 1

P_1

```
while(TRUE)
{
        while(turn ≠ 0);

            critical section

        turn = 1;

            reminder section

}
```

P_2

```
while(TRUE)
{
        while(turn ≠ 1);

            critical section

        turn = 0;

            reminder section

}
```

Two Processes: Solution – 1

- Mutual Exclusion: Satisfied.
- Progress: Not Satisfied.
  - ***P_1 and P_2 are running in Loop.***
  - `turn=0` and P_1 enters to its critical section.
  - P_1 makes `turn=1` in its exit section.
  - P_1 wishes to enter to critical section but can't.
- Bounded waiting: Satisfied

once

forced alternate

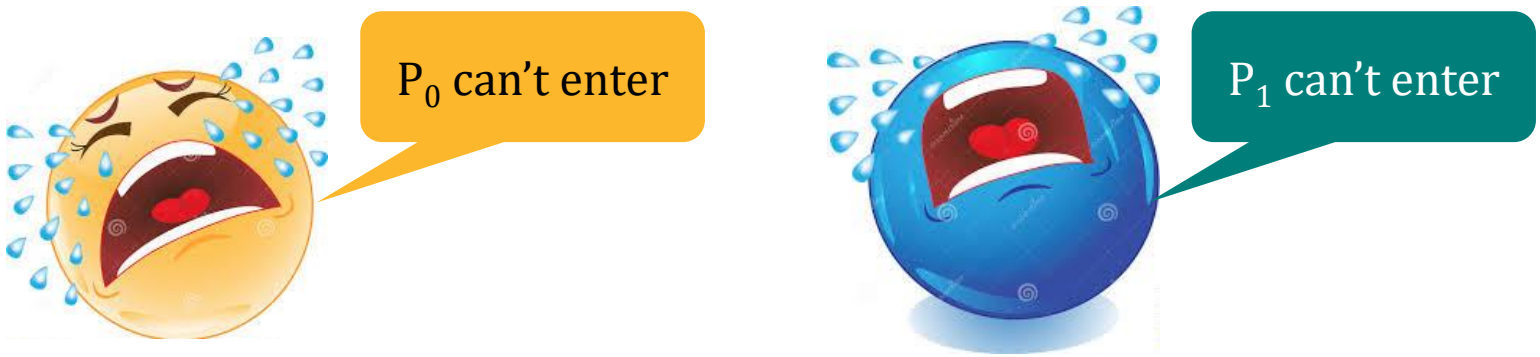
Two Processes: Solution – 2

- **`int flag[1]`** //initialized to **FALSE**

P_0	P_1
`do {`	`do {`
`flag[0] = TRUE;` **`while(flag[1]);`**	**`flag[1] = TRUE;`** **`while(flag[0]);`**
`critical section`	**`critical section`**
`flag[0] = FALSE;`	**`flag[1] = FALSE;`**
`reminder section`	`reminder section`
`} while(TRUE)`	`} while(TRUE)`

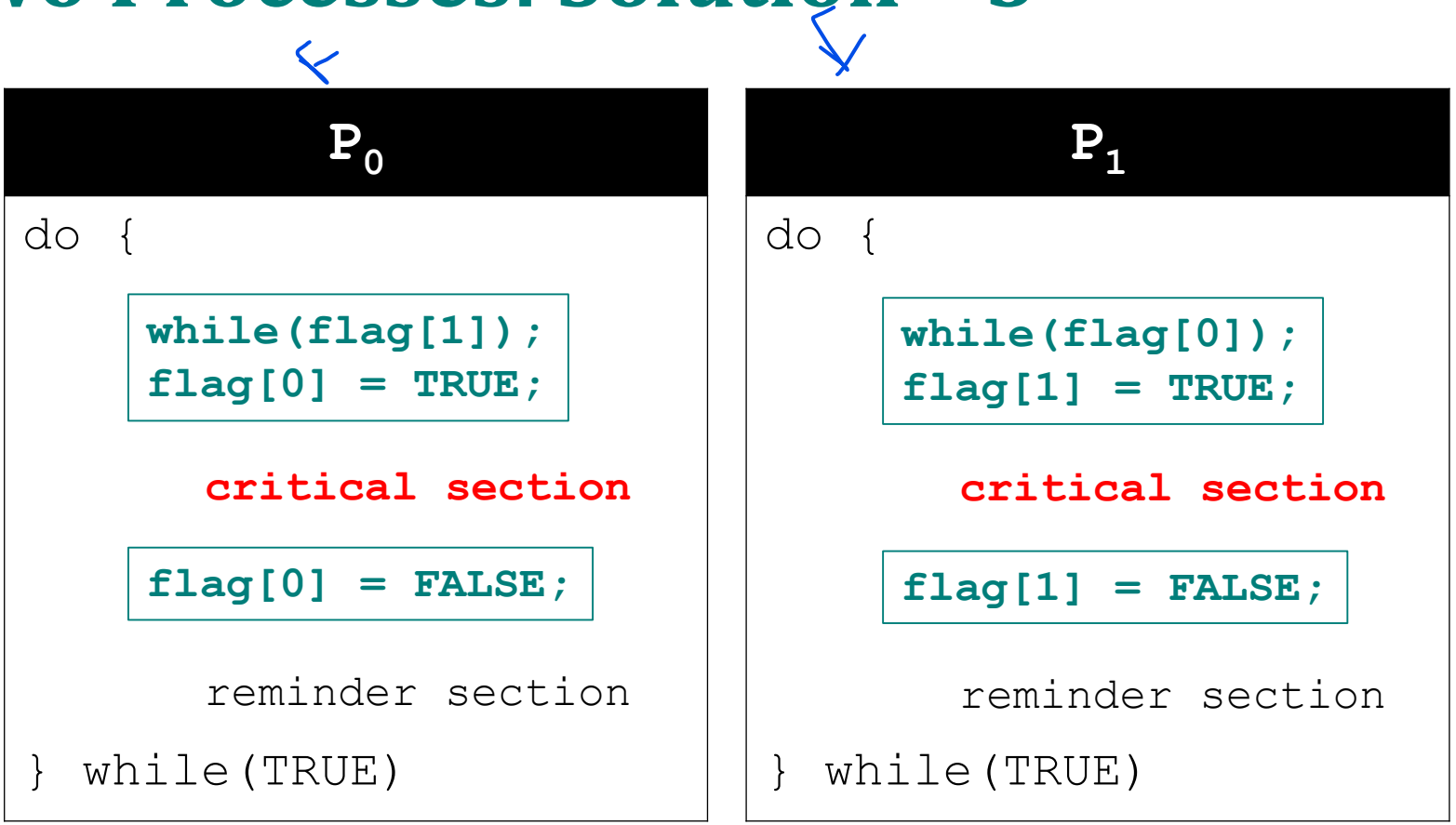
Two Processes: Solution – 2

- Mutual Exclusion: Satisfied.
- Progress: Not Satisfied.
  - P_0 makes `flag[0]=TRUE`
  - P_1 makes `flag[1]=TRUE`



- Bounded waiting: Satisfied

Two Processes: Solution – 3



Two Processes: Solution – 3

- Mutual Exclusion: Not Satisfied.
  - while(flag[1]); -- Pass
  - while(flag[0]); -- Pass
  - P_0 makes flag[0] = TRUE; and enters into critical section
  - P_1 makes flag[1] = TRUE; and enters into critical section
- Progress: Satisfied.
- Bounded waiting: Not Satisfied
  - P_0 may continuously enter into the critical section even when P_1 is waiting in its while loop.

Two Processes: Solution – 4 [Peterson's Solution]

P_0

```
do{
    flag[0] = TRUE;
    turn = 1;
    while(flag[1] && turn==1);

       critical section

    flag[0] = FALSE;

       reminder section
} while(TRUE)
```

P_1

```
do{
    flag[1] = TRUE;
    turn = 0;
    while(flag[0] && turn==0);

       critical section

    flag[1] = FALSE;

       reminder section
} while(TRUE)
```

Two Processes: Solution – 4 [Peterson's Solution]

- Mutual Exclusion: Satisfied.
- Progress: Satisfied.
- Bounded waiting: Satisfied

Multiple Processes Solution [Bakery Algorithm]

- ***Inspired:*** On a bakery, there are *2 ticket generating machines*, suppose. If *mutual exclusive* access is not given to the number then more than two processes (customers) may have same number. In case of tie, process with *lowest pid* will be served first.
  - **Other example:** Pizza centers, College canteens etc.
- (num1, pid1) < (num2, pid2)
  - True; if num1 < num2
  - True; if num1==num2 && pid1 < pid2

Multiple Process Solution (Original) [Bakery Algorithm]

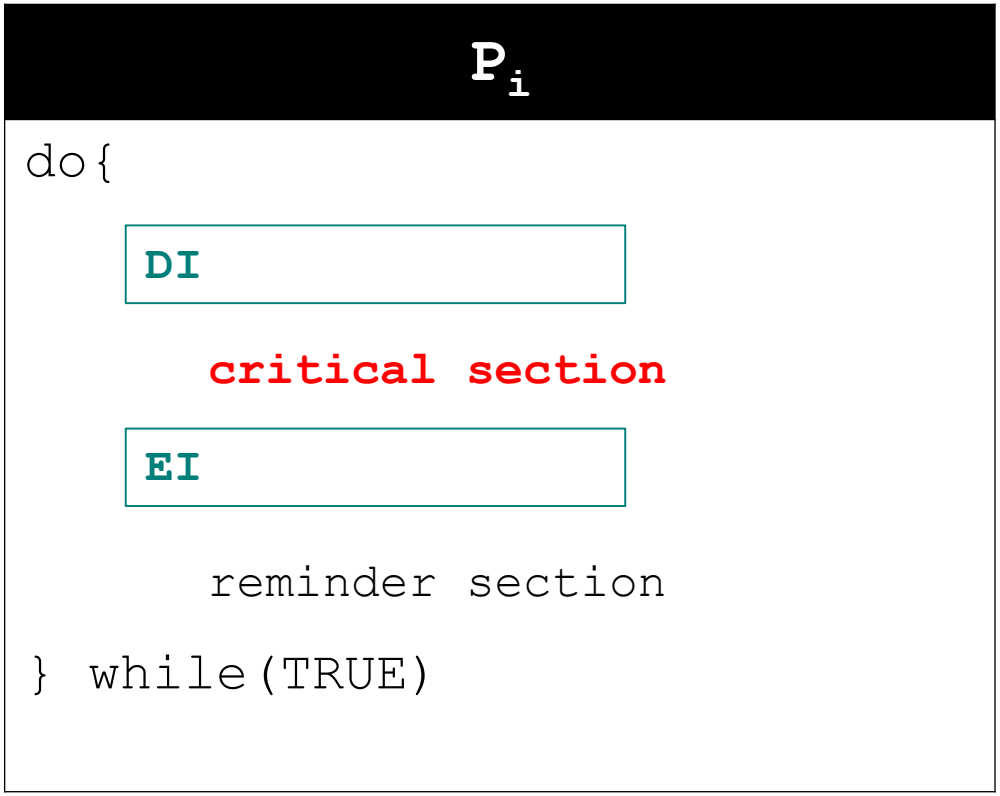
P_i

```
do{
    choosing[i] = TRUE;
    number[i]= max(number[0:n-1])+1
    choosing[i] =  FALSE;
    for j = 0 to n-1 {
       while(choosing[j]);
       while( number[j] && (number[j],j) < (number[i],i) );
    }

        critical section

    number[i]=0;

        reminder section

} while(TRUE)
```

Hardware Solutions to Critical Sections

- Enable and Disable Interrupt
- Test-and-set instruction
- Swap instruction

Enable and Disable Interrupt



- May miss out some important system interrupts
- Not suitable for multi-processor systems

Test-and-Set instruction

- Test-and-set instruction - defined below as if it were a function

```
boolean Test-and-Set (boolean *target){
   boolean rv = *target;  // return value
   *target = true;  // set value of target
   return(rv);
}
```

Test-and-Set instruction: Solution1

P_i

```
do{
    while(Test-and-Set(&lock));

        critical section

    lock = FALSE;

        reminder section
} while(TRUE)
```

- lock initialized to FALSE.
- Mutual exclusion: YES
- Progress: YES
- **Bounded waiting: NO**

Test-and-Set instruction: Solution2

- Variables used:
  - global boolean waiting[n] //initialized to FALSE
  - global boolean lock //initialized to FALSE
  - local key

Test-and-Set instruction: Solution2

P_i

```
do{
    waiting[i] = TRUE; key = TRUE;
    while(waiting[i] && key)
      key = Test-and-Set(&lock);
    waiting[i]=FALSE;

       critical section

    j = (i+1) % n;
    while(j!=i && waiting[j]==FALSE)
      j = (j+1) % n;
    if(j==i)
      lock = FALSE;
    else waiting[j] = FALSE;

       reminder section

} while(TRUE)
```

- Mutual exclusion: YES
- Progress: YES
- Bounded waiting: YES

Swap Instruction

- Swap instruction - defined below as if it were a function

```
boolean Swap (boolean *a, *b){
   boolean temp = *a;
   *a = *b;
   *b = temp;
}
```

Swap Instruction: Solution1

- global boolean lock;
- local boolean key;
- lock & key initialized to FALSE

- Mutual exclusion: YES
- Progress: YES
- **Bounded waiting: NO**

P_i

```
do{
    key = TRUE;
    while(key)
       Swap(&lock, &key);

      critical section

    lock = FALSE;

      reminder section
} while(TRUE)
```

P_j

```
do{
    key = TRUE;
    while(key)
       Swap(&lock, &key);

      critical section

    lock = FALSE;

      reminder section
} while(TRUE)
```

Swap instruction: Solution2

P_i

```
do{
    waiting[i] = TRUE; key = TRUE;
    while(waiting[i] && key)
      Swap(&lock, &key);
    waiting[i]=FALSE;

       critical section

    j = (i+1) % n;
    while(j!=i && waiting[j]==FALSE)
      j = (j+1) % n;
    if(j==i)
      lock = FALSE;
    else waiting[j] = FALSE;

       reminder section

} while(TRUE)
```

- Mutual exclusion: YES
- Progress: YES
- Bounded waiting: YES

Semaphore

- A synchronization primitive proposed by Dijkstra in 1968.
- Semaphores (S) are ***integer variables*** that are used to solve the critical section problem by using ***two atomic operations***, ***wait*** and ***signal*** that are used for process synchronization.
- P(S) and V(S) operations are ***atomic***.
  - Atomic ***means*** that *variable* on which *read*, *modify* and *update* happens at the same time/moment with no pre-emption i.e. in between *read*, *modify* and *update* ***no other operation is performed that may change the variable***.

Semaphore

- **Wait (S) -** The *wait operation* ***decrements*** the value of its argument S, if it is positive. ***If S is negative or zero***, then *no operation is performed*.
- **Signal (S) -** The *signal operation* ***increments*** the value of its argument S.

```
wait(S)
    {
       while (S<=0); //Busy wait
        S--;
    }
```

```
Signal (S)
    {
      S++
    }
```

Types of Semaphores

- **Counting Semaphores -** These are ***integer*** value semaphores and have an ***unrestricted value domain***. These semaphores are used to coordinate the resource access, where the semaphore count is the number of available resources.
  - If the resources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.
- **Binary Semaphores -** The binary semaphores are like counting semaphores but their value is restricted to 0 and 1.
  - The *wait operation* only works when the *semaphore is 1* and the *signal operation* succeeds when *semaphore is 0*. It is sometimes *easier* to implement *binary semaphores* than *counting semaphores*.

Process Execution using Semaphores

P_i

```
do
    {
        Wait (S)
        // Critical Section
        Signal (S)
        // Remainder Section
    } While (T)
```

- Mutual exclusion: YES
- Progress: YES
- **Bounded waiting: NO**

Application of Semaphores

- **Order of Execution:** 3 processes P1, P2, P3. Execution order should be P2→P1→P3

P1	**P2**	**P3**
Wait(S1)		Wait (S2)
P1	P2	P3
Signal (S2)	Signal (S1)	

- Two semaphores S1 = 0 and S2 = 0 are used.
- In case of context switch also, the order of execution would be P2→P1→P3

Application of Semaphores

- **Managing Resources:** In 5 printers how many processes can run?

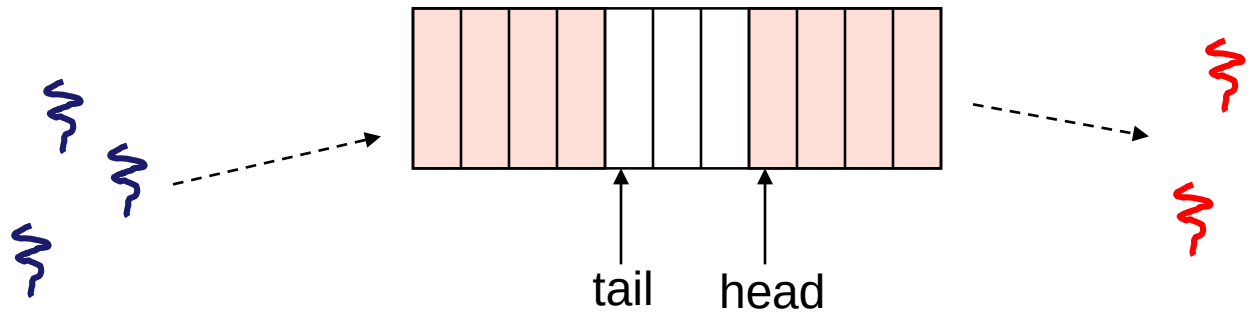
$$P_i$$

```
{
    Wait (S)
    // Critical Section
    Signal (S)
    // Remainder Section
}
```

- Start from S = 5.

Producer Consumer Problem (Bounded Buffer Problem)

- *AKA* "producer/consumer" problem
  - there is a buffer in memory with N entries
  - producer threads insert entries into it (one at a time)
  - consumer threads remove entries from it (one at a time)
- Threads are concurrent
  - so, we must use synchronization constructs to control access to shared variables describing buffer



Producer Consumer Problem

- Constraints
  - Only one thread can manipulate the buffer at a time **(mutual exclusion)**
  - The consumer (thread) must wait if buffers are empty **(synchronization constraint) (Underflow)**
  - The producer (thread) must wait if buffers are full **(synchronization constraint) (Overflow)**

Producer Consumer Problem

- Semaphore mutex = 1,
  empty = n, // All slots are empty initially
  full = 0 // Initially, all slots are empty.

Producer

```
do{

//produce an item

wait(empty);
wait(mutex);

//place in buffer

signal(mutex);
signal(full);

}while(true)
```

Consumer

```
do{

wait(full);
wait(mutex);

// remove item from buffer

signal(mutex);
signal(empty);

// consumes item

}while(true)
```

Readers-Writers Problem

- Consider a situation where we have a file shared between many people.
- If one of the people tries editing the file, no other person should be reading or writing at the same time, otherwise changes will not be visible to him/her.
- However if some person is reading the file, then others may read it at the same time.
- In OS, we call this situation as the **readers-writers problem**

Readers-Writers Problem

Problem Statement:

- One set of data is shared among a number of processes.
- Once a writer is ready, it performs its write. Only one writer may write at a time.
- If a process is writing, no other process can read it.
- If at least one reader is reading, no other process can write.
- Readers may not write and only read.

Solution when Reader has the Priority over Writer

- Priority means, no reader should wait if the share is currently opened for reading.
- Three variables are used: **mutex, wrt, readcnt** to implement solution.
- **semaphore** mutex, wrt; // semaphore **mutex** is used to ensure mutual exclusion when **readcnt** is updated i.e. when any reader enters or exit from the critical section and semaphore **wrt** is used by writers
- **int** readcnt; // **readcnt** tells the number of processes performing read in the critical section, initially 0.

Solution when Reader has the Priority over Writer

Writer process

```
do {
    // writer requests for critical section
    wait(wrt);

    // performs the write

    // leaves the critical section
    signal(wrt);

} while(true);
```

Solution when Reader has the Priority over Writer

Reader Process

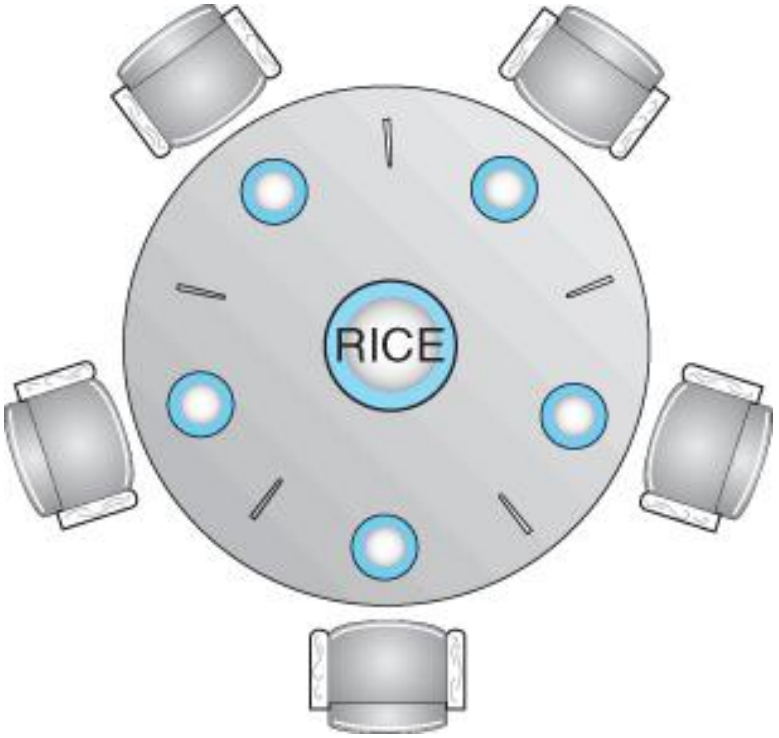
```
do {
        wait(mutex);
        readcnt++;
        if (readcnt==1)
             wait(wrt);
        signal(mutex);
              reader performs reading here
        wait(mutex);
         readcnt--;
         if (readcnt == 0)
                 signal(wrt);
         signal(mutex);
} while(true);
```

Solution when Writer has the Priority over the reader

Assignment

Dining Philosopher Problem

- The dining philosophers problem is a classic synchronization problem involving the *allocation of limited resources* amongst a group of processes in a *deadlock-free* and *starvation-free* manner.



- Consider *five philosophers* sitting around a table, in which there are *five chopsticks* evenly distributed and an *endless bowl of rice* in the center.
- There is ***exactly one chopstick*** between each pair of dining philosophers.

Dining Philosopher Problem: Conditions

- These philosophers spend their lives alternating between two activities: ***eating and thinking***.
- When it is time for a philosopher to eat, it must first acquire two chopsticks - one from their *left* and one from their *right*.
  - One at a time.
  - No fighting.
- When a philosopher *thinks*, it *puts down both chopsticks* in their original locations.

Dining Philosopher Problem

```
do {
   wait(chopstick[i]);
   wait(chopstick[(i+1) % 5]);
      . . .
   // eat
      . . .
   signal(chopstick[i]);
   signal(chopstick[(i+1) % 5]);
      . . .
   // think
      . . .
}while (TRUE);
```

The structure of philosopher i

CASE 1 – Philosopher come in order

- When the philosophers are coming in serially, one possible solution can be ***code shown earlier***.
- It uses a set of five semaphores (chopsticks[5]), and to have each hungry philosopher first wait on their left chopstick (chopsticks[i]), and then wait on their right chopstick (chopsticks[(i+1)%5]).
- ***If they decide to come in order then no problem will arise***.

CASE 2 – Race Condition

- What if, before a philosopher (process) can pickup right chopstick the next philosopher comes.
- This is a ***race condition***.
- To solve this an array of semaphores is needed.
- It is a special case where we allow more than one process to run.
  - In CS problem, only one process is allowed to run.

CASE 3 – Deadlock

- Suppose that all five philosophers get hungry at the same time, and each starts by picking up their *left chopstick*.
- They then look for their *right chopstick*, but because it is unavailable, they wait for it, forever, and eventually all the philosophers starve due to the resulting ***deadlock***.

Potential solutions to Deadlock:

- Only allow ***four philosophers*** to dine at the same time. (Limited simultaneous processes.)
  - Not Feasible as the problem has changed
- Allow philosophers to pick up chopsticks only when *both are available*, in a critical section. **(All or nothing allocation of critical resources)**
- Use an *asymmetric solution*, in which odd philosophers pick up their left chopstick first and even philosophers pick up their right chopstick first. (Will this solution always work? What if there are an even number of philosophers?)

Potential solutions to Deadlock:

- Note that a deadlock-free solution to the dining philosophers problem does not *necessarily guarantee a starvation-free one*.
- While some or even most of the philosophers may be able to get on with their normal lives of eating and thinking, there may be *one unlucky soul* who never seems to be able to get both chopsticks at the same time.

Thank You

Monitor vs Semaphore

Limitations

- One of the biggest limitation of semaphore is priority inversion.
- Deadlock, suppose a process is trying to wake up another process which is not in sleep state.Therefore a deadlock may block indefinitely.
- The operating system has to keep track of all calls to wait and to signal the semaphore.

Problem in this implementation of semaphore

- Whenever any process waits then it continuously checks for semaphore value (look at this line while (s==0); in P operation) and waste CPU cycle. To avoid this another implementation is provided below.

Binary Semaphore: Spin-Lock/Busy-Wait Solution

- Can take two values: 1 or 0 (initialized to 1)

```
Struct Semaphore{ int value; };
Semaphore mutex;
mutex.value=1;
```

P_i

```
do{
    while(mutex.value == 0);
    mutex.value=0;
        critical section
    mutex.value++;
        reminder section
} while(TRUE)
```

P(mutex)

V(mutex)

- Mutual exclusion: YES
- Progress: YES
- **Bounded waiting: NO**

Binary Semaphore: Solution with Waiting List

```
Struct Semaphore{
   int value;
   Struct Process *List;
};
Semaphore mutex;
mutex.value=1;
```

- Mutual exclusion: YES
- Progress: YES
- Bounded waiting: YES

P_i

```
do{
   if(mutex.value == 0){
      Add Pi to mutex->List;
      block();
   }
   else mutex.value=0;

      critical section

   if(mutex->List is nonempty){
      Remove Pj from mutex->List;
      wakeup(Pj);
   }
   else mutex.value++;

      reminder section
} while(TRUE)
```

Counting Semaphore

- Useful when we have multiple instances of same shared resource.
- Initialized to the number of instances of the resource. (e.g. printer)

Counting Semaphore: Spin-Lock/Busy-Wait Solution

```
Semaphore countSem;
countSem.value=3;
```

P_i

```
do{
    while(countSem.value == 0);
    countSem.value--;

      critical section

    countSem.value++;

      reminder section

} while(TRUE)
```

- Mutual exclusion: YES
- Progress: YES
- **Bounded waiting: NO**

Counting Semaphore: Solution with Waiting List

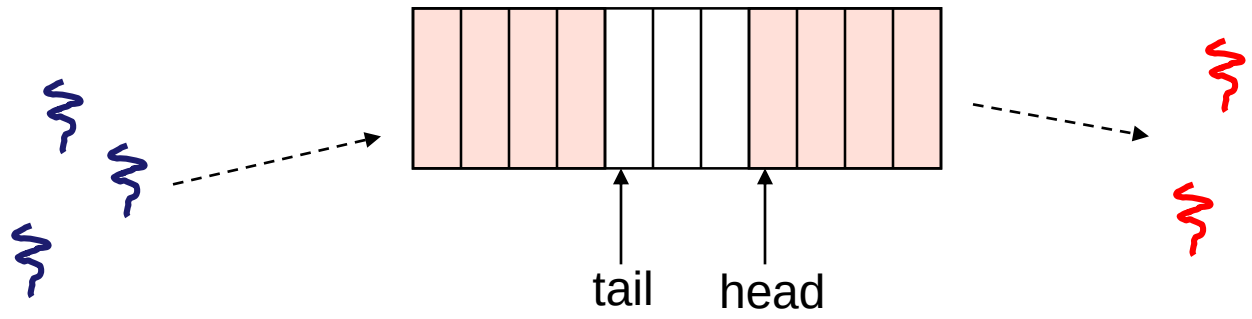
P_i

```
do{
    countSem.value--;
    if(countSem.value < 0){
      Add Pi to countSem->List;
      block();
    }

       critical section

    countSem.value++;
    if(countSem.value <= 0){
      Remove process Pj from countSem->List;
      wakeup(Pj);
    }

       reminder section
} while(TRUE)
```

Mutual exclusion: YES
Progress: YES
Bounded waiting: YES

Bounded Buffer Problem

- *AKA* "producer/consumer" problem
  - there is a buffer in memory with N entries
  - producer threads insert entries into it (one at a time)
  - consumer threads remove entries from it (one at a time)
- Threads are concurrent
  - so, we must use synchronization constructs to control access to shared variables describing buffer



Bounded Buffer Problem

- Constraints
  - The consumer must wait if buffers are empty (synchronization constraint)
  - The producer must wait if buffers are full (synchronization constraint)
  - Only one thread can manipulate the buffer at a time (mutual exclusion)

Bounded Buffer Problem: Developing a Solution

- Each constraint needs a semaphore

```
Semaphore mutex = 1;
Semaphore nFreeBuffers = N;
Semaphore nLoadedBuffers = 0;
```

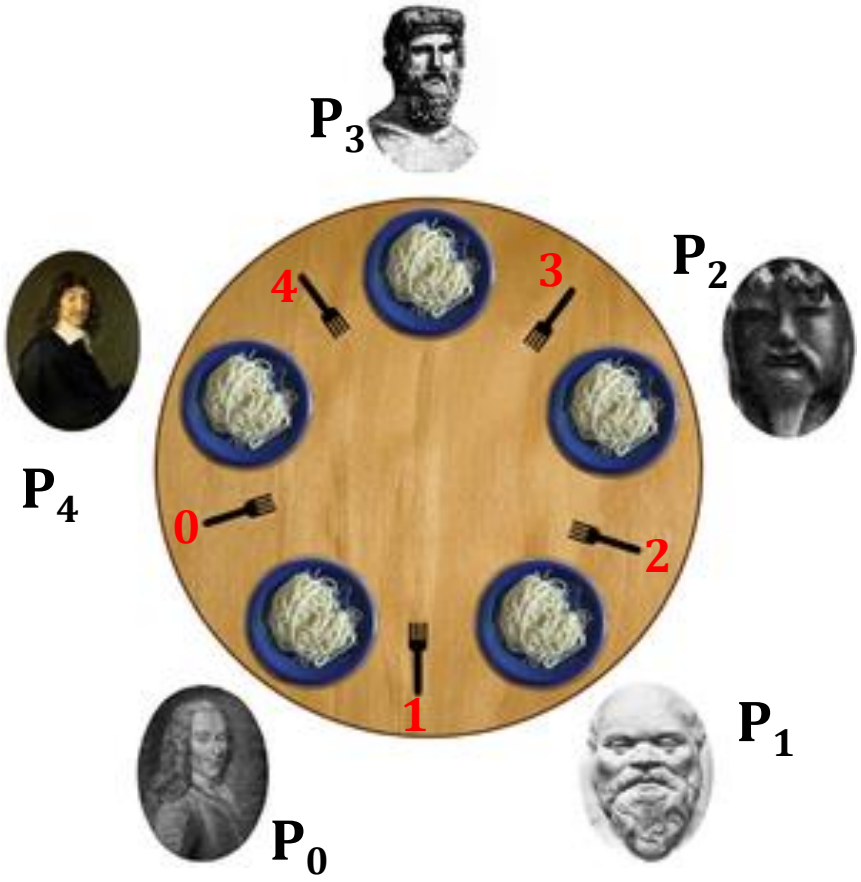
Producer

```
P(nFreeBuffers);
P(mutex);
// put 1 item in the buffer
V(mutex);
V(nLoadedBuffers);
```

Consumer

```
P(nLoadedBuffers);
P(mutex);
// take 1 item from buffer
V(mutex);
V(nFreeBuffers);
```

Dining Philosophers Problem



P_i

```
do{
    P(Chopstick[i])
    P(Chopstick[(i+1)%5])

    EAT

    V(Chopstick[(i+1)%5])
    V(Chopstick[i])

    THINK
} while(TRUE)
```

- What if all the philosophers grab their left chopsticks!!

Dining Philosophers Problem: Solution to deadlock

- Allow at most *n-1* philosophers to seat.
- Allow a philosopher to grab the chopsticks only if both are available.

```
DI
P(Chopstick[i])
P(Chopstick[(i+1)%5])
EI
```

- An odd philosopher grabs his left chopstick first then the right chopstick. An even philosopher grabs his right chopstick then the left chopstick.

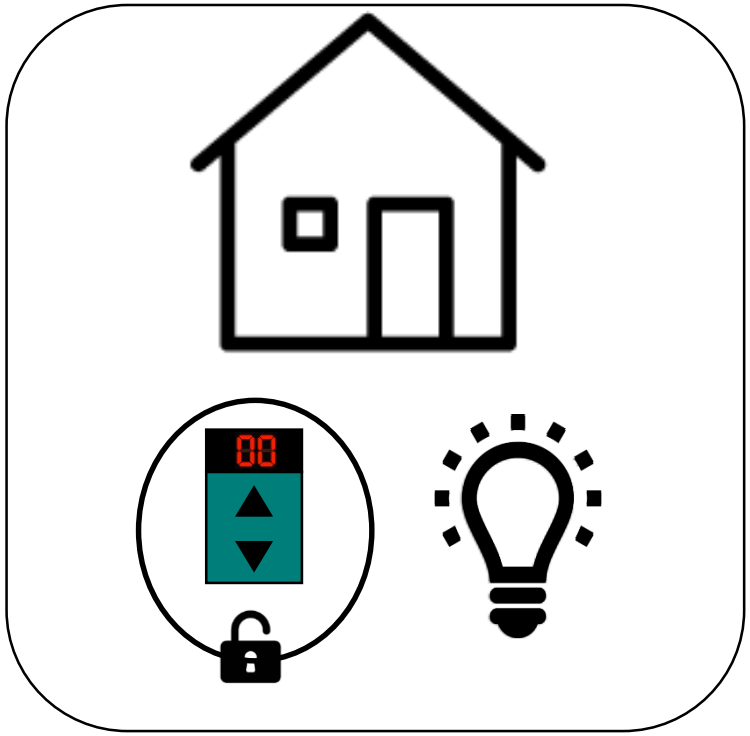
P_0	P_1	P_2	P_3	P_4
P(C[1])	P(C[1])	P(C[3])	P(C[3])	P(C[0])
P(C[0])	P(C[2])	P(C[2])	P(C[4])	P(C[4])

Readers Writers Problem

- A data set is shared among a number of concurrent processes
  - Readers – only read the data set; they do not perform any updates
  - Writers – can both read and write.
- Conditions
  - Any number of readers may simultaneously read the file.
  - If a writer is writing to the file, no reader/writer is allowed to access the file.

Readers Writers Problem: Solution 1 – Preference to Readers

- Semaphore mutex initialized to 1.
- Integer rdcount initialized to 0.
- Semaphore wSem initialized to 1.



R_i

```
do{
    P(mutex);
     rdcount++;
     if (rdcount==1)   P(wSem);
    V(mutex);

        READ

    P(mutex);
     rdcount--;
     if (rdcount==0)   V(wSem);
    V(mutex);

} while(TRUE)
```

W_i

```
do{
    P(wSem);

      WRITE

    V(wSem);
} while(TRUE)
```

Readers Writers Problem: Solution 1 – Preference to Readers

- Readers only
  - All readers are allowed to READ
- Writers only
  - One writer at a time
- Both readers and writers with read first
  - Writer has to wait on P(wrt)
- Both readers and writers with write first
  - Reader has to wait on P(wrt)

Writers may starve !

Readers Writers Problem: Solution 2 – Preference to Writers

- Integer **`rdcount`** – keeps track of number of readers.
- Integer **`wrtcount`** – keeps track of number of writers.
- Semaphore **`mutex1`** – controls the updating of rdcount.
- Semaphore **`mutex2`** – controls the updating of wrtcount.
- Semaphore **`rSem`** – inhibits all readers while there is at least one writer desiring access to critical section.
- Semaphore **`wSem`** – inhibits all writers while there is at least one reader desiring access to critical section.
- Semaphore **`mutex3`** – to avoid long queue on rSem. So that waiting writer processes get preference.

Readers Writers Problem: Solution 2 – Preference to Writers

R_i

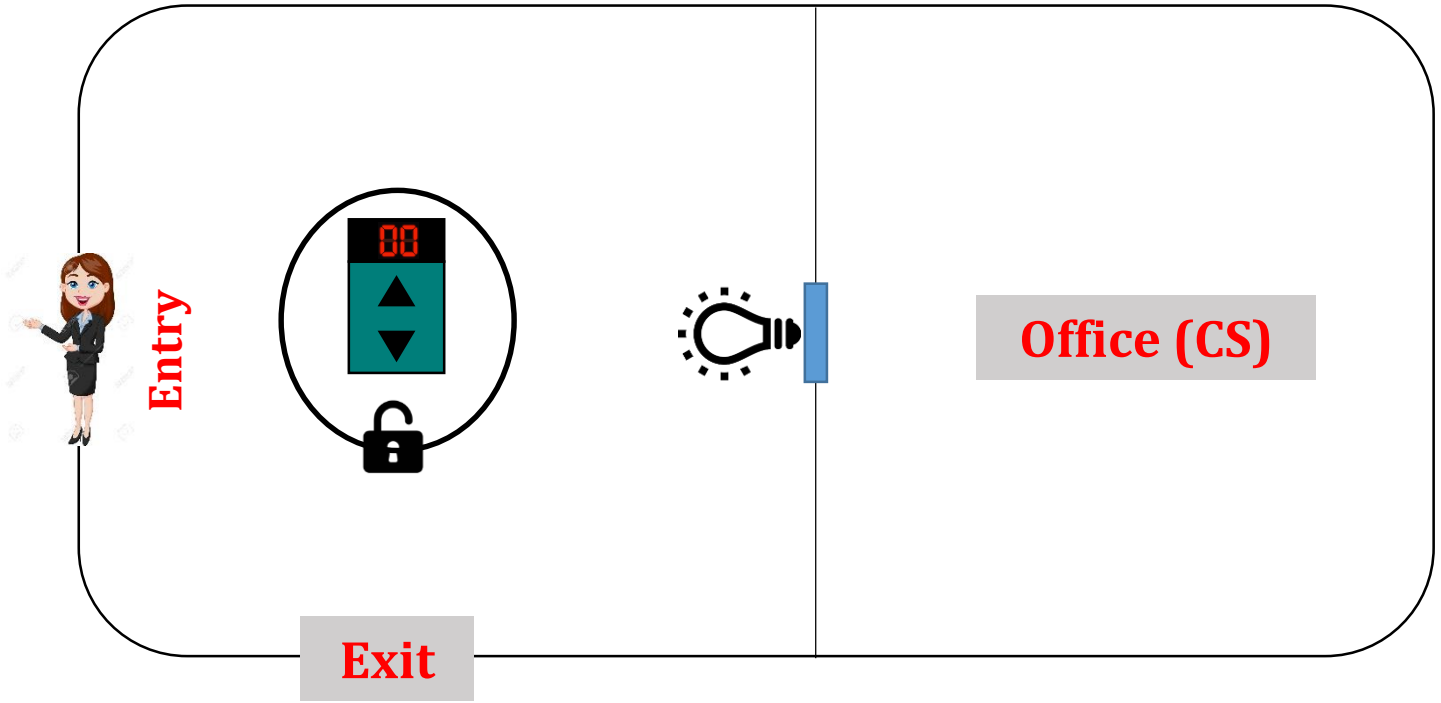
```
do{
    P(mutex3);
     P(rSem);
      P(mutex1);
       rdcount++;
       if(readcount == 1)   P(wSem);
      V(mutex1);
     V(rSem);
    V(mutex3);

      READ

    P(mutex1);
     readcount--;
     if (readcount == 0)   V(wSem);
    V(mutex1);

      reminder section
} while(TRUE)
```

W_i

```
do{
    P(mutex2);
     wrtcount++;
     if(writecount == 1)  P(rSem);
    V(mutex 2);

    P(wSem);

      WRITE

    V(wSem);

    P(mutex2);
     writecount--;
     if (writecount == 0) V(rSem);
    V(mutex 2);

      reminder section
} while(TRUE)
```

Readers may starve !

Readers Writers Problem: Solution 3 – Based on the arrival order



- Integer **rdcount**
- Semaphore **rdcount_mutex**
- Semaphore **access_mutex**
- Semaphore **order_mutuex**

Readers Writers Problem: Solution 3 – Based on the arrival order

R_i

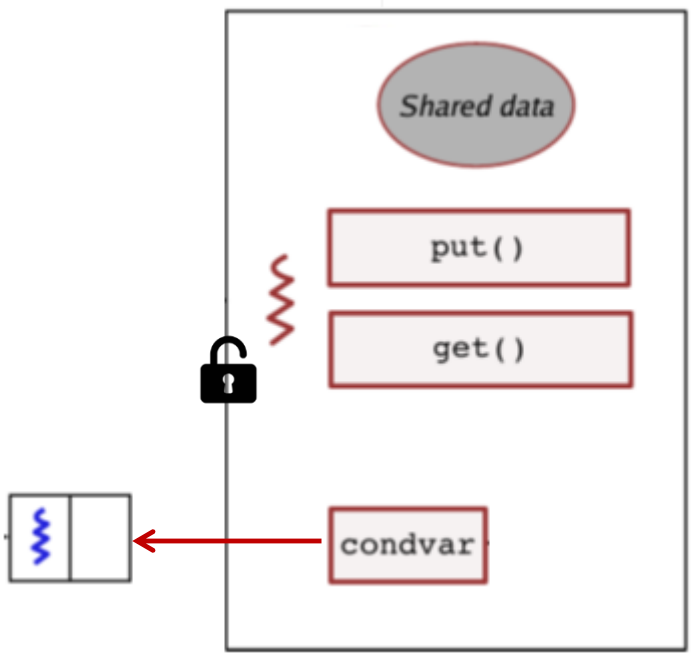
```
do{
    P(order_mutex);
     P(rdcount_mutex);
      rdcount++;
      if (rdcount==1)
         P(access_mutex));
     V(rdcount_mutex);
    V(order_mutex);

       READ

    P(rdcount_mutex);
     rdcount--;
     if (rdcount==0)
        V(access_mutex);
    V(rdcount_mutex);

} while(TRUE)
```

W_i

```
do{
    P(order_mutex);
     P(access_mutex));
    V(order_mutex);

      WRITE

    V(access_mutex);

} while(TRUE)
```

Monitor: A structured synchronization tool

- A monitor is a module that encapsulates:
  - some shared data
  - some atomic procedures
  - a set of condition variables



- Only one process at a time may be active in a monitor

Monitor: A structured synchronization tool

- Condition variables allow for blocking and unblocking.
  - If a thread/process has to wait/block for some event to occur by some other thread/process, then it waits in the queue of the corresponding condition variable. e.g., **condvar.wait()**.

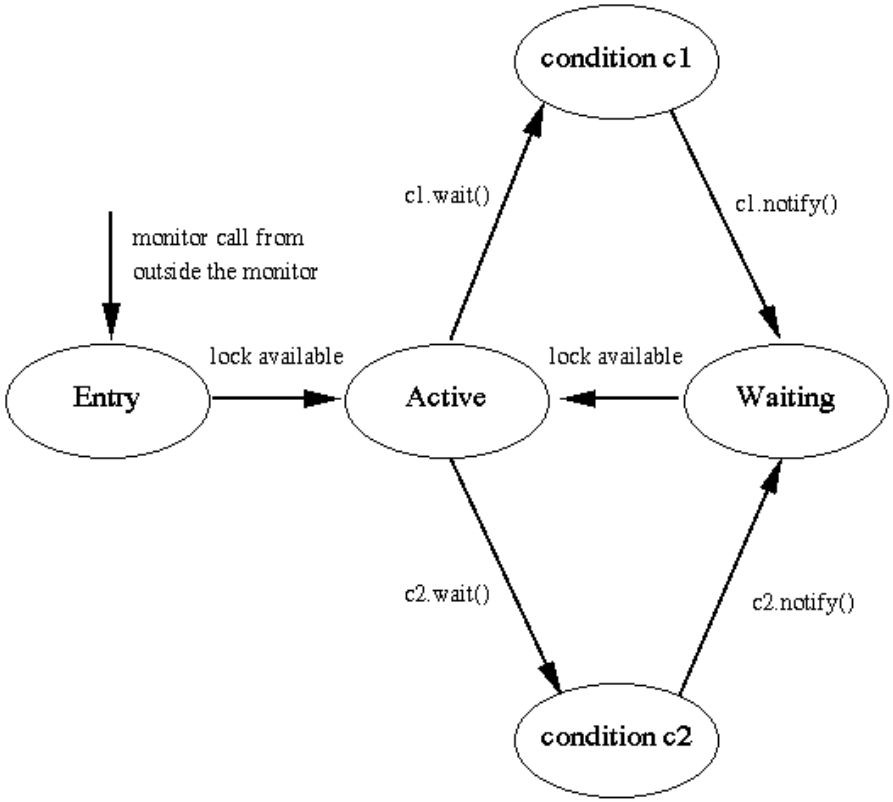
    (Note: it immediately releases the monitor lock)
  - If a thread/process has generated the event, it can indicate this by executing **condvar.signal() or condvar.notify()**.
    - This wakes up a waiting thread/process from condvar queue.
    - If condvar queue is empty then it cond.signal() doesn't do anything.
    - What happens to the signaler thread/process?
      - Implementation 1: They wait in the signaler queue
      - Implementation 2: They are active (no signaler queue)

Queues in monitor implementation

- The **entry queue** contains processes attempting to call a monitor procedure from outside the monitor.
  - Each monitor has one entry queue.
- The **waiting queue** contains processes that have been awakened by a notify operation.
  - Each monitor has one waiting queue.
- **Condition variable queues** contain processes that have executed a condition variable wait operation.
  - There is one such queue for each condition variable.
- The **signaler queue** contains processes that have executed a notify/signal operation.
  - Each monitor has at most one signaler queue.
  - In some implementations, a notify leaves the process active and no signaler queue is needed.

Monitor state transition:
Without signaler queue

- The lock becomes available when the active process executes a wait or leaves the monitor.



Solution to Producer Consumer problem using Monitor

```
monitor PC {
    int DATA[10];
    int R, F;
    Condition FULL, EMPTY;
    Produce(v) {
        if ( (R+1)%10== F ) then FULL.Wait();
        put v into DATA array;
        EMPTY.Signal();
    }
    Consume() {
        if( R == 0 ) then EMPTY.Wait();
        consume the next DATA array value;
        FULL.Signal();
    }
   init(){ R=F=0; }
}
```

Producer$_i$

```
…
…
Produce(25);
…
…
```

Consumer$_i$

```
…
…
Consume(25);
…
…
```

Monitor in Java

One modern language that uses monitors is Java.

- Each object has its own monitor.
- Methods are put in the monitor using the synchronized keyword.
- Each monitor has one condition variable.
- The methods on the condition variables are: wait(), notify(), and notifyAll().
- Since there is only one condition variable, the condition variable is not explicitly specified.

Java program without synchronization

```
public class ThreadA {
    public static void main(String[] args) {
        ThreadB b = new ThreadB();
        b.start();

        System.out.println("Total is: " + b.total);

    }
}

class ThreadB extends Thread {
    int total;

    @Override
    public void run() {
        for (int i = 0; i < 100; i++) {
            total += i;
        }
    }
}
```

// The result would be 0, 10, etc. Because sum is not finished before it is used.

Java program with synchronization

```
public class ThreadA {
    public static void main(String[] args){
        ThreadB b = new ThreadB();
        b.start();

        synchronized(b){
                System.out.println("Waiting for b to complete...");
                b.wait();

                System.out.println("Total is: " + b.total);
        }
    }
}

class ThreadB extends Thread{
    int total;
    @Override
    public void run(){
        synchronized(this){
            for(int i=0; i<100 ; i++){
                total += i;
            }
            notify();
        }
    }
}
```

Output
Waiting for b to complete...
Total is: 4950